

Introduction to Programming

Course Code: CSE1001

MODULE 1 Complete

By Ms Savathallapalli Jasmine

Module no.	Syllabus/Topic	Course Outcome
1	Introduction to components of computer system, Types of Computer, Introduction to Programming, Algorithm and Flowchart, Overview of C language, Basic structure, Data concepts in C: Constants, Variables, Expressions, Errors, Operators, Data Types And Statements: Declarations, Input-Output Statements, Compound statements, Selection Statements. Conditions, Operators and its precedence, Branching and Looping statements.	CO1 Understand the structured programming language fundamentals

Introduction to Computer System

Computer System

Computer is an electronic device for storing and processing data, typically in binary form, according to instructions given to it in a variable program.

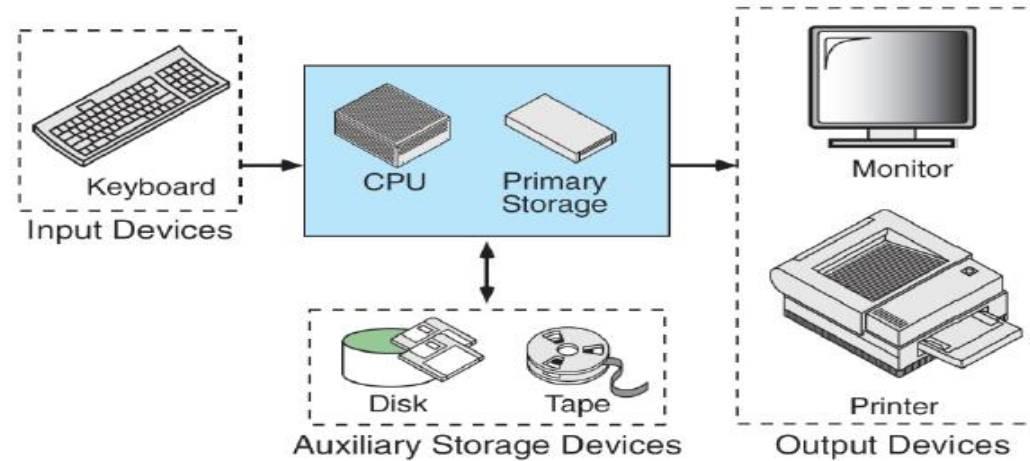
Main task performed by a computer

- Accept the data
- Process or manipulate the data
- Display or store the result in the form of human understanding
- Store the data, instructions and results.

Characteristics of Computers

- **High Processing Speed** : Computers can process data and perform calculations at incredibly high speeds.
- **Accuracy** : Computers perform operations with a high degree of accuracy. Errors usually occur due to human mistakes (like wrong input), not the machine.
- **Reliability** : Computers consistently perform tasks without tiring or losing efficiency.
- **Versatility** : A computer can perform a wide range of tasks — from playing music and games to performing scientific simulations.
- **Diligence** : Unlike humans, computers do not suffer from fatigue or lack of concentration. They can perform repeated tasks without boredom or errors due to tiredness.

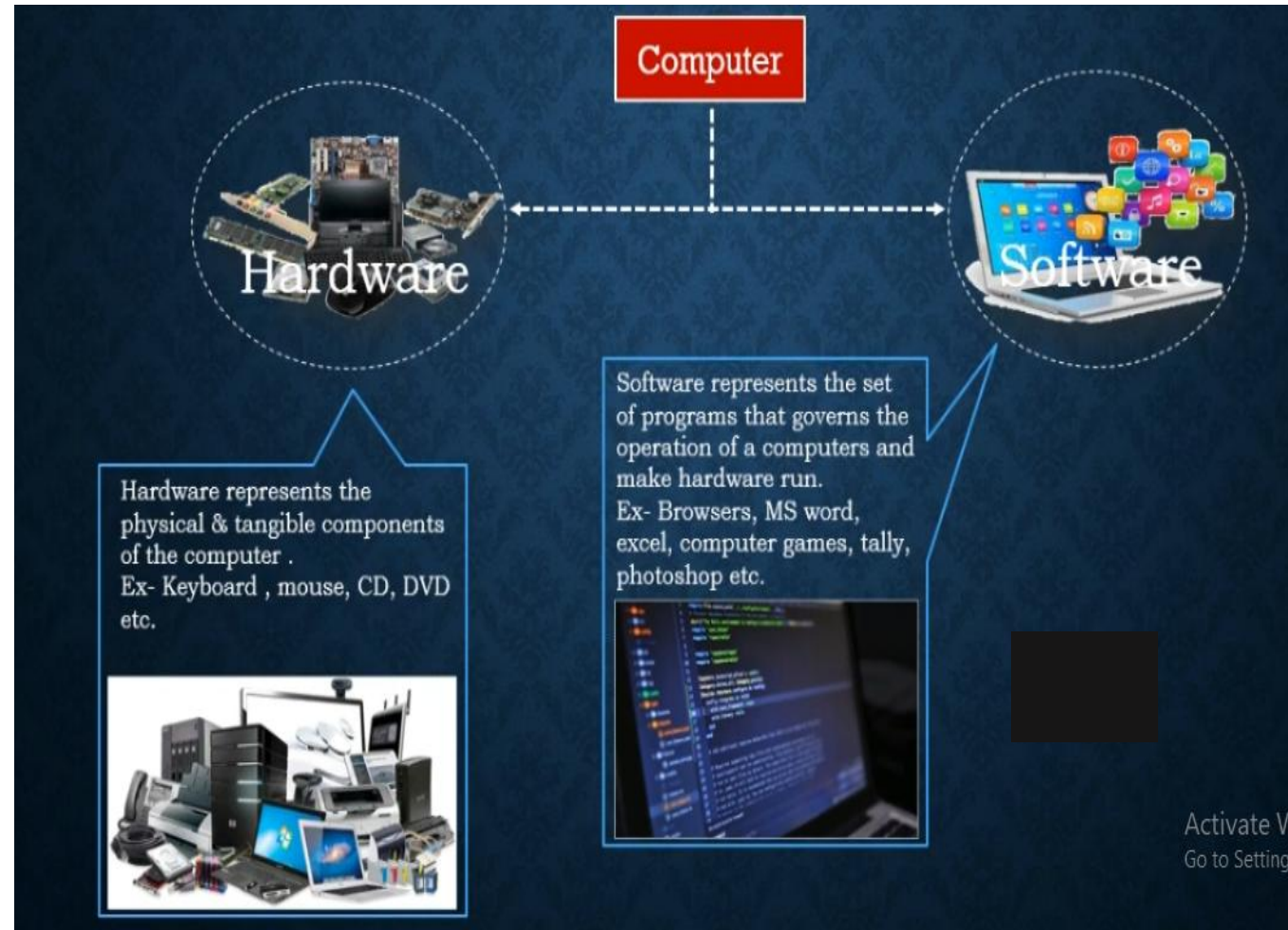
Block Diagram of Computer System



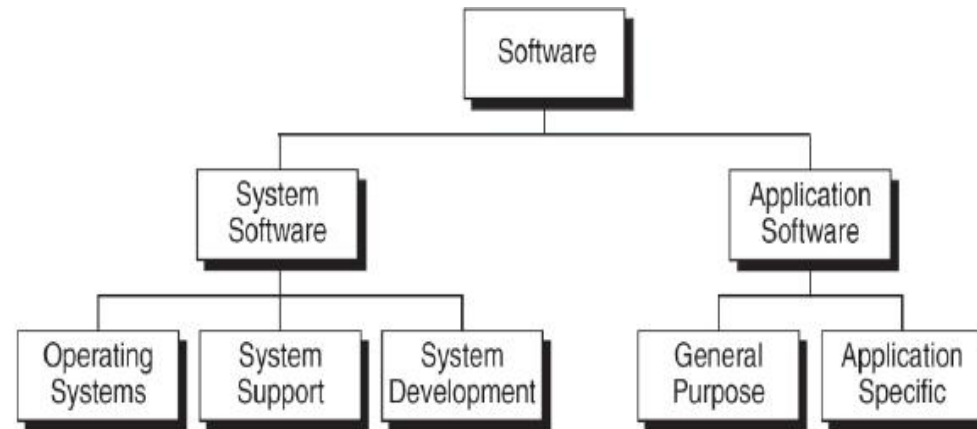
Computer hardware is the collection of physical elements that comprise a computer system. The hardware component of the computer system consists of five parts:

1. Input Devices
2. CPU
3. Primary Memory
4. Output devices
5. Storage devices

Hardware and Software



Hardware and Software



Computer software is a collection of computer programs and related data that provides the instructions for a computer what to do and how to do.

Software are further categories into two:

System Software: System software is responsible for managing a variety of independent hardware components, so that they can work together.

Application Software: Application software is developed to aid in any task that benefits from computation

System Software

- ☉ The software that controls internal computer operations (like reading data from input devices, sending processed information to output devices, checking system components etc.) is known as **system software**.
- ☉ System software coordinates the activities and functions of the hardware and software. In addition, it controls the operations of the computer hardware and provides an **environment or platform** for all the other types of software to work in.
- ☉ The best-known example of system software is the **operating system** (OS), which manages all the other programs in a computer.
- ☉ System software can further be classified into :
 - Operating System
 - Language Processors
 - Device Driver
 - Utilities software

OPERATING SYSTEM

- The **operating system (OS)** is the most important program that runs on a computer.
- Operating system itself decides : How to do? What to do? And When to do?
- The primary goal of an operating system is *to make the computer system convenient to use* and secondary goal is to use computer hardware in an efficient manner.
- Every general-purpose computer must have an operating system to run other programs and applications.
- Commonly used OS are :



Activate Windows
Go to Settings to activate Wi

Functions of Operating System

- **Provide an Interface**
- **Managing Files**
- **Managing peripherals and drivers**
- **Managing Memory**
- **Managing multi-tasking**
- **Managing Interrupts**
- **Providing a platform for run applications**
- **Managing user accounts**

- **System support software** provides system utilities and other operating services. Examples of system utilities are sort programs and disk format programs. Operating services consists of programs that provide performance statistics for the operational staff and security monitors to protect the system and data.
- **System development software**, includes the language translators that convert programs into machine language for execution, debugging tools to ensure that the programs are error free and computer.

Application Software

Application Software: Application software is developed to aid in any task that benefits from computation. This category includes:

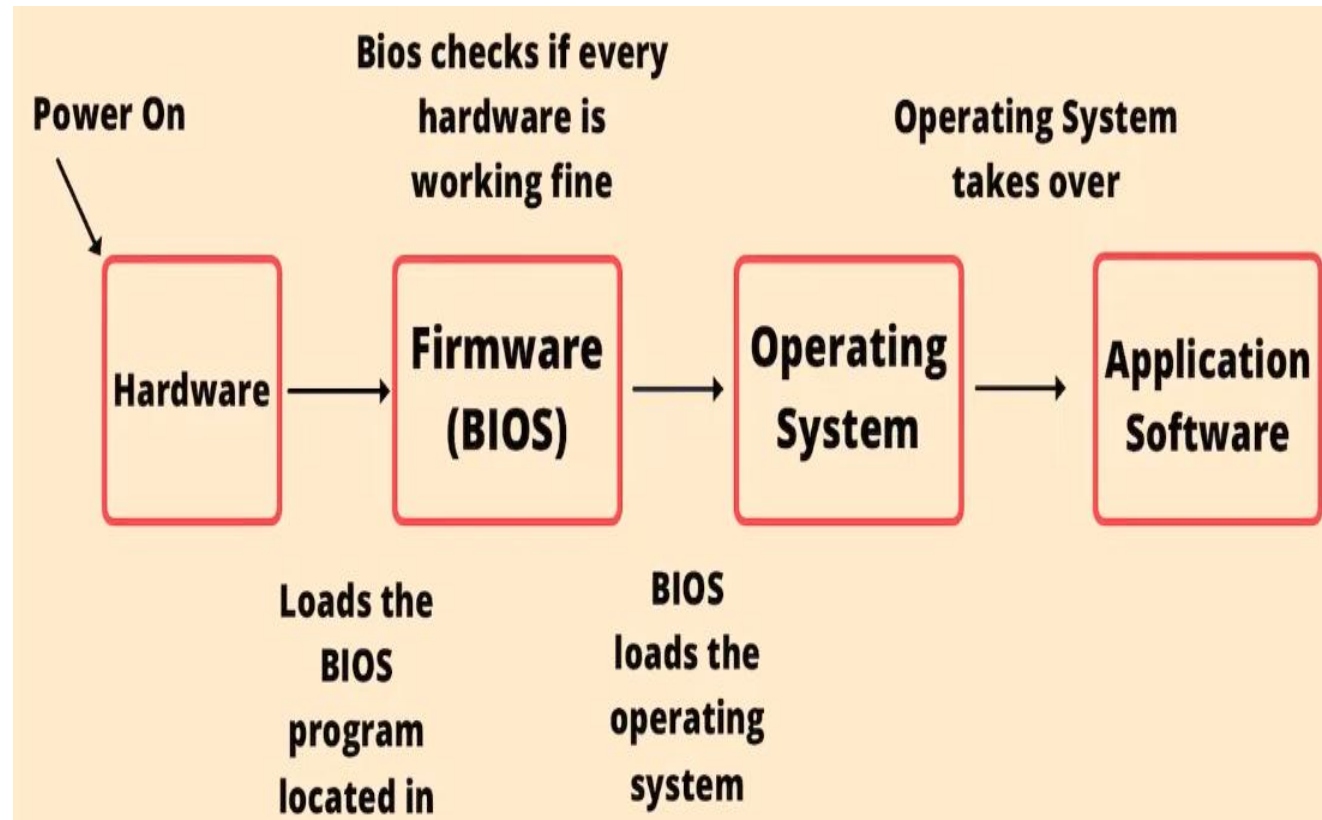
- Business software
- Computer aided design
- Databases
- Decision making software
- Educational software
- Image editing

Application software is broken in to two classes: general-purpose software and application –specific software.

General purpose software is purchased from a software developer and can be used for more than one application. Examples of general-purpose software include word processors, database management systems, and computer aided design systems. They are labelled general purpose because they can solve a variety of user computing problems.

Application –specific software can be used only for its intended purpose.

HCI (Human Computer Interaction)



TYPES OF COMPUTER

Category	Type	Key Features / Uses	Examples
Based on Size	Supercomputer	Fastest, trillions of instructions/sec, used in simulations, research	Weather forecasting, nuclear research
	Mainframe Computer	Supports 1000s of users, high storage, used in large organizations	Banking, telecom
	Minicomputer	Medium size, supports 4–200 users, cheaper than mainframe	Institutes, billing, accounting
	Workstation	High-performance, single-user, used in technical/scientific tasks	CAD, animation, data analysis
	Personal Computer (PC)	General-purpose, small, individual use	Laptop, desktop
	Server Computer	Stores & manages data/resources, runs 24/7	Web servers, database servers

TYPES OF COMPUTER

Based on Data Handling	Analog Computer	Processes continuous data (approx. values)	Speedometer, thermometer
	Digital Computer	Processes binary (0 & 1), accurate & fast	Laptop, desktop, smartphone
	Hybrid Computer	Combines analog + digital, accurate & real-time	Petrol pump processor, ECG
Modern Devices	Tablets & Smartphones	Portable, touchscreen, wireless, run apps	iPad, Android phones

Programming Languages

In order to communicate with the computer user also needs to have a language that should be understood by the computer.

Languages are divided into three categories-

1940's - Machine level Languages

1950's – Middle Language

1960's - High-Level Languages

Low Level Languages : Low level computer languages are machine codes or close to it. Computer cannot understand instructions given in high level languages or in English. It can only understand and execute instructions given in the form of machine language i.e. language of 0 and 1

Middle Level (Assembly) Language

The programs are written using the instructions of CPU is called Assembly level language. It uses symbols, or mnemonics, to represent the various machine language instructions. Examples are TASM, MASM

Programming Languages

TASM and **MASM** are both assemblers—programs that convert assembly language code into machine code—but they were developed by different companies and have some differences in syntax and usage.

TASM (Turbo Assembler)

Developer: Borland (founded by Philippe Kahn)

Platform: Primarily for **MS-DOS** systems

MASM (Microsoft Macro Assembler)

Developer: Microsoft

Platform: MS-DOS and Windows (later versions support 32-bit and 64-bit)

Programming Languages

High Level Languages:

High-level languages are designed to relieve the programmer from the details of the assembly language. High level languages share one thing with symbolic languages, They must be converted into machine language. The process of converting them is known as compilation.

Speaking Almost Directly to the Machine

Machine Language

The only language a CPU executes natively

```
10110000 01100001  
10100011 00000000
```

- Written in pure binary (0s & 1s)
- Executes with no translation needed
- Completely hardware-dependent
- Extremely difficult to write & debug

Assembly Language

Human-readable symbols for machine code

```
MOV AL, 61h  
ADD AL, BL  
JMP LOOP
```

- Uses mnemonics (MOV, ADD, JMP)
- Needs an assembler to convert to binary
- One step above machine code
- Still tied to a specific processor

HIGH-LEVEL LANGUAGES

Designed for People, Translated for Machines

High-level languages read closer to English and math notation. A compiler or interpreter translates the code into machine instructions, which makes these languages portable across different hardware.

1

Portability

Same source code can run on many machines with little or no change.

2

Readability

Uses English-like keywords: if, while, print, function.

3

Faster Development

Less code needed to express the same logic than assembly.

4

Needs Translation

Requires a compiler or interpreter before/while it runs.

COMMON EXAMPLES

C

C++

Java

Python

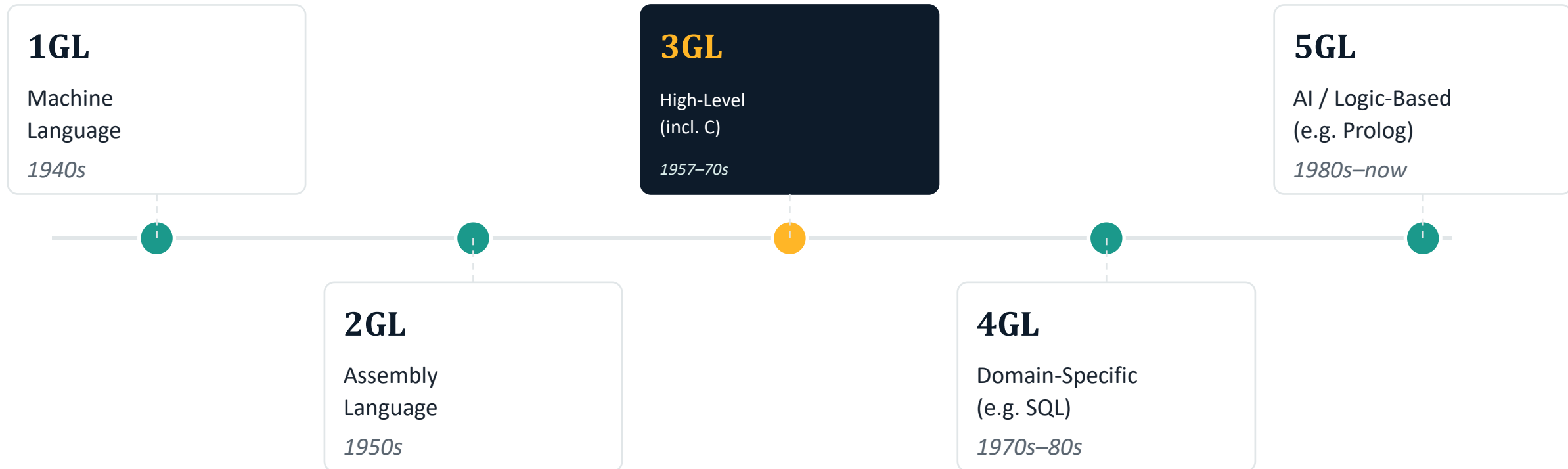
JavaScript

Low-Level vs. High-Level Languages

Aspect	Low-Level Language	High-Level Language
Closeness to hardware	Very close	Abstracted away
Readability	Difficult (binary / mnemonics)	Easy, English-like syntax
Execution speed	Very fast	Slower (needs translation)
Portability	Machine-specific	Portable across platforms
Development time	Long, complex	Short, productive
Typical examples	Machine code, Assembly	C, Python, Java

Five Generations of Programming Languages

Beyond the low/high-level split, languages are grouped into five generations that trace their historical evolution from raw binary to AI-driven, goal-based programming.



1GL and 2GL: The Foundation Layers

1GL Machine Language

FORM	Pure binary (0s and 1s)
ERA	1940s, earliest computers
EXECUTION	Runs directly on the CPU — no translation
SPEED	Fastest possible execution
DRAWBACK	Unreadable, error-prone, not portable

2GL Assembly Language

FORM	Mnemonic codes (MOV, ADD, SUB)
ERA	1950s, symbolic programming begins
EXECUTION	Converted to machine code by an assembler
SPEED	Very fast, near machine-level performance
DRAWBACK	Still processor-specific, tedious to write

3GL — Where C Enters the Picture

Third-generation languages introduced structured, procedural programming that is portable across machines. C, created by Dennis Ritchie at Bell Labs in 1972, became the defining 3GL — powerful enough for system programming yet readable enough for everyday application code.

```
hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!");
    return 0;
}
```

Why C Matters

Built Unix. Influenced C++, Java, C#, JavaScript, and more. Still used today in operating systems, embedded devices, and compilers.



Procedural

Code organized into functions and step-by-step logic.



Portable

Same source compiles for many hardware platforms.



Close to Hardware

Supports pointers & direct memory access.



Structured

Encourages loops, conditionals, modular design.

4GL and 5GL: Toward Higher Abstraction

4GL

Domain-Specific Languages

- Built for a specific task, e.g. database queries or report generation
- Examples: SQL, MATLAB, R
- Much less code than 3GL for the same result
- Trades general-purpose flexibility for productivity

5GL

AI & Constraint-Based Languages

- Programmer states the goal; the system finds the how
- Examples: Prolog, Mercury, OPS5
- Used in AI, expert systems, and logic programming
- Still evolving — closest to natural human instruction

Translators

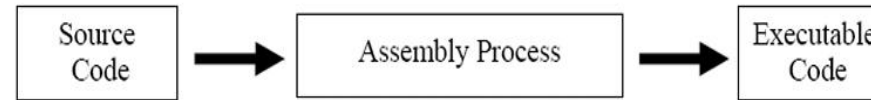
High level and Mid-level languages require translators for communicating the instructions to the computer.
We have three types of translators:

Assembler

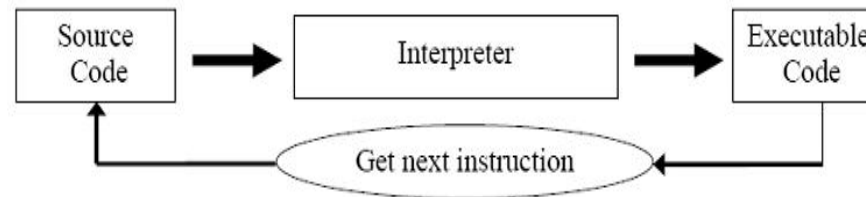
Interpreter

Compiler

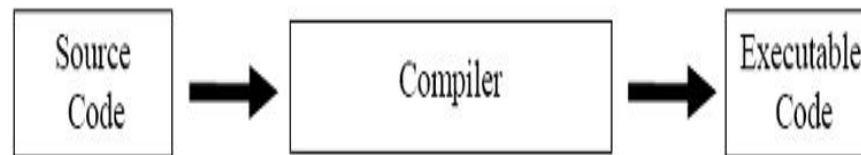
Assembler: An Assembler converts an assembly program into machine code.



Interpreter: An Interpreter is also a program that translates high-level source code into executable code. However, the difference between a compiler and an interpreter is that an **interpreter translates one line at a time and then executes it**, and then the next instruction is translated. This goes on until the end of the program.



Compiler: A Compiler is a program that translates a high-level language into machine code in one go.



Comparison between a Compiler and Interpreter

COMPILER	INTERPRETER
A Compiler is used to compile an entire program and an executable program is generated through the object program	An interpreter is used to translate each line of the program code immediately as it is entered
The executable program is stored in a disk for future use or to run it in another computer	The executable program is generated in RAM and the interpreter is required for each run of the program
The compiled programs run faster	The Interpreted programs run slower
Most of the Languages use compiler	A very few languages use interpreters.

Source Code: Source code is the code that is **input to a translator**.

Executable Code: Executable code is the code that is **output from the translator**

ALGORITHM

An algorithm is a description of a procedure which terminates with a result. Algorithm is a step-by-step method of solving a problem.

Properties of an Algorithm:

- 1) **Finiteness:** - An algorithm terminates after a finite number of steps.
- 2) **Definiteness:** - Each step-in algorithm is unambiguous. This means that the action specified by the step cannot be interpreted in multiple ways & can be performed without any confusion.
- 3) **Input:** - An algorithm accepts zero or more inputs
- 4) **Output:-** An algorithm should produce at least one output.
- 5) **Effectiveness:** - It consists of basic instructions that are realizable. This means that the instructions can be performed by using the given inputs in a finite amount of time.

Algorithm

A step-by-step plan to solve a problem, written in plain language

- An Algorithm is a set of clear steps to solve a problem
- Written in simple English (or pseudocode) — NOT in a programming language
- Good Algorithm should be: Clear, have a Finite number of steps, and give the Correct output
- We design the algorithm FIRST, then convert it into C code
- Example task: add two numbers entered by the user



Algorithm: Add Two Numbers

Step 1: Start

Step 2: Read two numbers a and b

Step 3: Calculate $\text{sum} = a + b$

Step 4: Print sum

Step 5: Stop

Flowcharts

Graphical or symbolic representation of an algorithm is Flow chart.

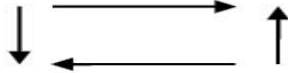
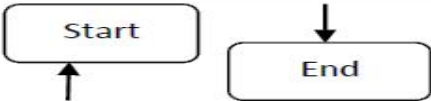
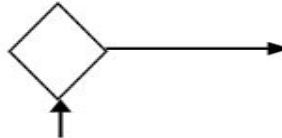
Diagrammatic representation of the step-by-step solution to a given problem

Advantages of Flowchart:

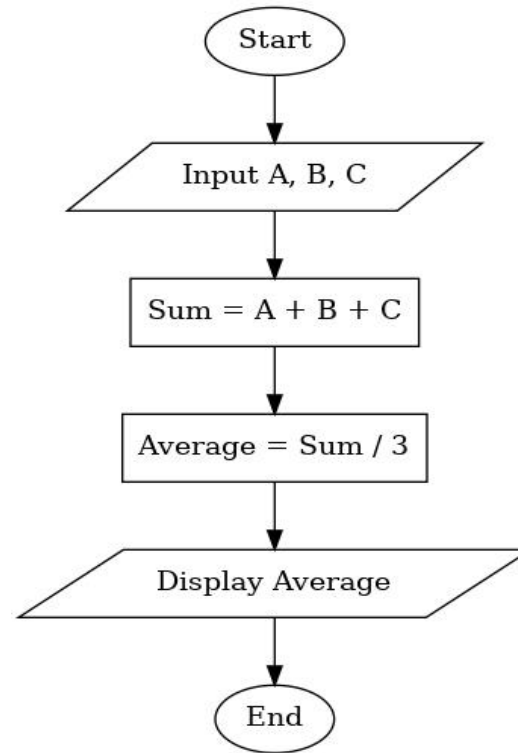
- 1. Conveys Better meaning:** Since a flowchart is a pictorial representation of an algorithm, it is easier for a programmer to understand and explain the logic of the program to other programmers.
- 2. Analyses the problem effectively:** Flow chart helps the programmers to analyze the problem in detail by breaking the flow chart into detailed parts.
- 3. Effective Coding:** It is very easy for the programmers to write the program, because flow chart will give a clear idea of the steps involved.
- 4. Systematic Debugging :** It enables easy detection of errors.

Disadvantages:

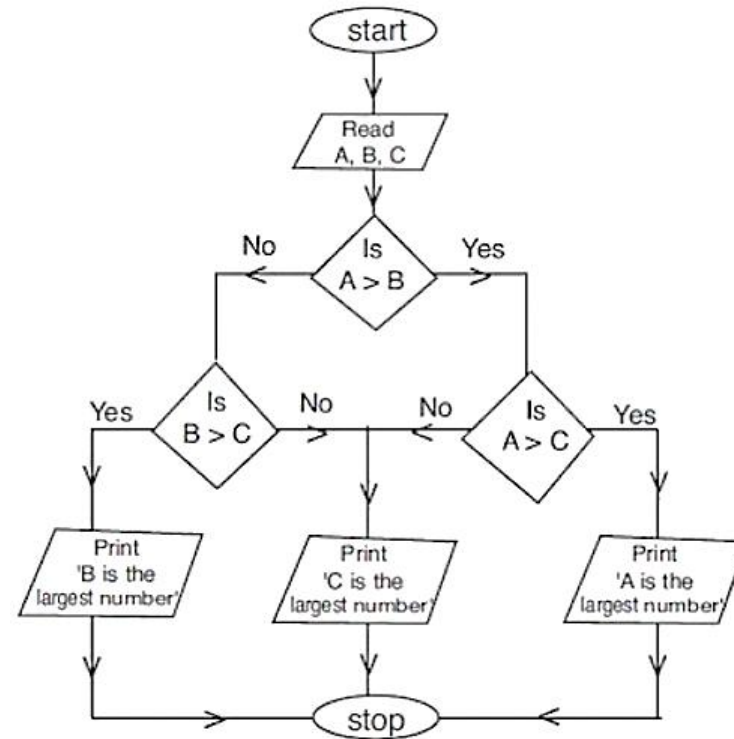
1. For a large program flowchart becomes very complex and confusing.
2. Modification of a flowchart is difficult.
3. Excessive use of connectors may confuse programmers.

S.NO	Description	Symbols
1	Flowlines : These are the left to right or top to bottom lines connection symbols. These lines shows the flow of control through the program.	
2	Terminal Symbol : The oval shaped symbol always begins and ends the flowchart. Every flow chart starting and ending symbol is terminal symbol.	
3	Input / Output symbol : The parallelogram is used for both input (Read) and Output (Write) is called I/O symbol. This symbol is used to denote any function of an I/O device in the program.	
4	Process Symbol : The rectangle symbol is called process symbol. It is used for calculations and initialization of memory locations.	
5	Decision symbol : The diamond shaped symbol is called decision symbol. This box is used for decision making. There will be always two exists from a decision symbol one is labeled YES and other labeled NO.	
6	Connector Symbol (circle) : This is a Circular-shaped symbol used to connect different parts of flowchart. When the flow chart is lengthy, it is split into different pages. Then these connectors are used to connect between these pages at the beginning and at the end of each page.	

Flowchart for average of three numbers



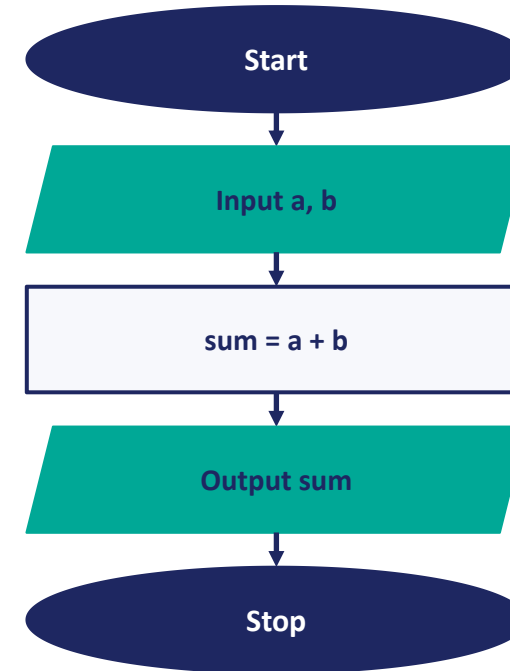
Flowchart for largest of three numbers



Flowchart

A picture (diagram) that shows the steps of an algorithm using symbols

- A Flowchart draws the algorithm using easy-to-read symbols and arrows
- Oval — Start / Stop
- Parallelogram — Input / Output
- Rectangle — Process (a calculation or action)
- Diamond — Decision (Yes / No question)
- Arrows show the ORDER in which steps happen



Why use a flowchart? It helps us SEE the logic of a program clearly before writing any code, and makes it easy to spot mistakes.

Practice Questions

1. What are the main tasks performed by a computer system?
2. Explain any five characteristics of computers with examples?
3. Differentiate between hardware and software?
4. What is an algorithm? Explain its properties.
5. What is a flowchart? Write two advantages and two disadvantages.

INTRODUCTION TO C LANGUAGE

History of C

C is a general-purpose high level language that was originally developed by Dennis Ritchie for the Unix operating system. It was first implemented on the Digital Equipment Corporation PDP-11 computer in 1972.

The Unix operating system and virtually all Unix applications are written in the C language. C has now become a widely used professional language for various reasons.

1. Easy to learn
2. Structured language
3. It produces efficient programs.
4. It can handle low-level activities.
5. It can be compiled on a variety of computers.

Facts about C

- C was invented to write an operating system called UNIX.
- C is a successor of B language which was introduced around 1970
- The language was formalized in 1988 by the American National Standard Institute (ANSI).
- By 1973 UNIX OS almost totally written in C.
- Today C is the most widely used System Programming Language. C is fast, efficient, close to hardware, and portable, making it a preferred choice for performance-critical and system-level applications.
- Most of the state of the art software have been implemented using C

Why to use C?

C was initially used for system development work, in particular the programs that make-up the operating system. C was adopted as a system development language because it produces code that runs nearly as fast as code written in assembly language. Some examples of the use of C might be:

- Operating Systems
- Language Compilers
- Assemblers
- Text Editors
- **Databases:** MySQL, PostgreSQL.

Language	Year	Developed By
ALGOL	1960	International Group
BCPL	1967	Martin Richard
B	1970	Ken Thompson
Traditional C	1972	Dennis Ritchie
K & R C	1978	Kernighan & Dennis Ritchie
ANSI C	1989	ANSI Committee
C89	1990	ISO
C99	1999	Standardization committee

In 1983, ANSI (American national standard institute) established a committee, whose goal was to produce a machine independent definition of the C language.

The committee approved a version of c language known as ANSI C in 1989, then it also approved by ISO the result is C89.

In 1999 the standardization committee released a version of C99 by adding some features of C++ / JAVA.

Overview of C Language

- **What is C?**

A general-purpose, structured programming language. It is called a "middle-level" language because it can work close to hardware and also write normal applications.

- **Who made it?**

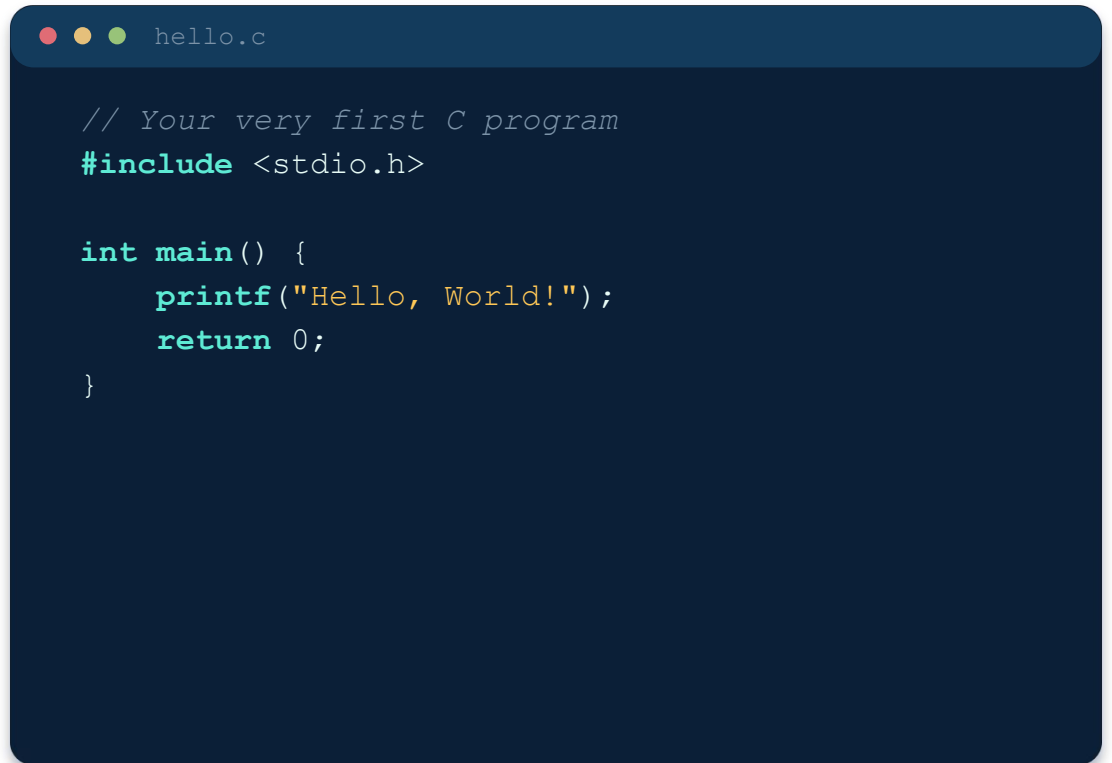
Created by Dennis Ritchie in 1972 at Bell Labs, to build the UNIX operating system.

- **Why learn C?**

It is easy to learn, fast, portable, and forms the base for languages like C++, Java, and Python.

- **Where is it used?**

Operating systems, compilers, embedded systems, game engines, and device drivers.



```
// Your very first C program
#include <stdio.h>

int main() {
    printf("Hello, World!");
    return 0;
}
```

Output: Hello, World!

The Journey to C: Languages Before C

Year	Language	Developed By	Contribution
1960	ALGOL 60	International committee	Introduced structured, block-based programming concepts
1966	BCPL (Basic CPL)	Martin Richards, Cambridge	Simple, machine-independent systems language
1970	B Language	Ken Thompson, Bell Labs	Simplified BCPL further; used to write early UNIX
1972	C Language	Dennis Ritchie, Bell Labs (AT&T)	Added data types & structures to B; became the systems language for UNIX

Characteristics/ Features of C Language

Modularity: Ability to break down large modules into a manageable sub modules.

Example: calculator can be divided into arithmetic and trigonometric and again that is divided into sub modules.

Extendibility: Ability to extend an existing software by adding new features is called extendibility. It is a by-product of modularity.

Portability: Ability to port (install) existing software in different platforms (Unix, dos etc.) is called portability

Efficiency and speed: C language has rich set of data types and operators making the language more faster and efficient.

Flexibility: C compiler has only 32 keywords. With 32 keywords a programmer is able to solve any type of task, due to this reason programmers have the complete control over the language their by gaining the flexibility.

Why C? Features & Applications

KEY FEATURES

- Simple & structured (functions/blocks)
- Fast — compiles close to machine code
- Portable — 'write once, compile anywhere'
- Rich set of operators & built-in functions
- Supports pointers — direct memory access
- Extensible via user-defined functions
- Middle-level language — bridges low & high level

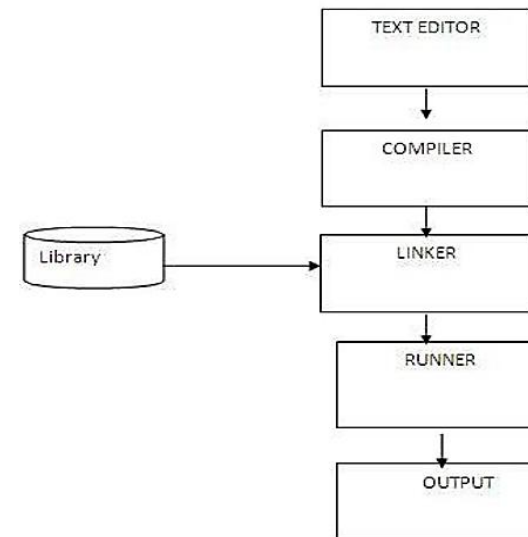
REAL-WORLD APPLICATIONS

- Operating systems — UNIX, Linux kernel, parts of Windows
- Embedded systems — microwaves, ATMs, routers
- Compilers & interpreters for other languages
- Database engines — MySQL core
- Game engines & performance-critical software
- Foundation language taught before C++, Java, Python

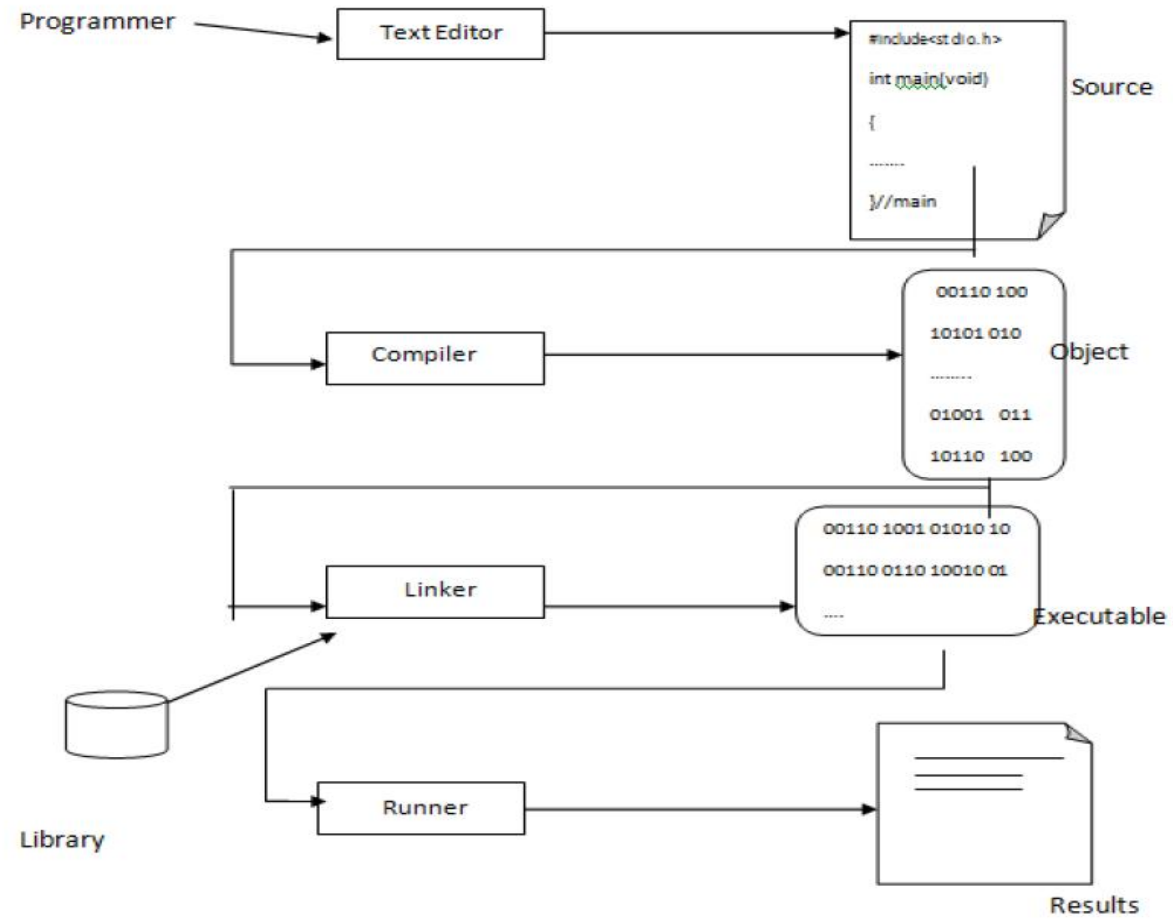
CREATING AND RUNNING PROGRAMS

- **Writing and Editing Programs:** Programs are written using a text editor and saved as source files with C code.
- **Compiling Programs:** The compiler (preprocessor + translator) converts source code into machine-readable object code.
- **Linking Programs:** The linker combines user-defined and library functions into a single executable file.
- **Executing Programs:** The loader loads the executable into memory, and the OS runs the program, handling input, processing, and output.

Sl. No.	Phase	Name of Code	Tools	File Extension
1	TextEditor	Source Code	C Compilers Edit, Notepad Etc..,	.C
2	Compiler	Object Code	C Compiler	.OBJ
3	Linker	Executable Code	C Compiler	.EXE
4	Runner	Executable Code	C Compiler	.EXE



PROCESS OF COMPILING AND RUNNING C PROGRAM



Basic Structure of a C Program

The skeleton every C program follows, in order

- 1 **Documentation Section**
Comments describing the program (optional)
- 2 **Header File Section**
#include statements — bring in ready-made functions
- 3 **Global Declaration**
Variables/functions visible to the whole file
- 4 **main() Function**
Program execution always starts here
 - 5 **Declaration Part**
Variables used inside main() are declared
 - 6 **Executable Part**
The actual statements that do the work
- 7 **User-Defined Functions**
Extra functions written by the programmer

```
structure.c

// 1. Documentation section
#include <stdio.h> // 2. Header file

int total; // 3. Global declaration

int main() { // 4. main() function
    int a = 5; // -- declaration part
    total = a + 10; // -- executable part
    printf("%d", total);
    return 0;
}

int square(int x) { // 5. User-defined func
    return x * x;
}
```

Output: 15

Sections of a C Program — Explained

- **Documentation section**

Comments (`//` or `/* */`). Ignored by the compiler; they only help humans understand the code.

- **Header file section**

Files like `stdio.h` contain ready-made functions. `#include` copies their declarations into your file before compiling.

- **Global declaration**

Variables declared outside every function. They can be used by `main()` and by any other function.

- **`main()` function**

Every C program MUST have exactly one `main()`. Execution always begins here, regardless of where other functions are written.

- **Declaration + Executable parts**

Inside `main()`, variables are declared first, then statements that use them are written and run in order.

- **User-defined functions**

Reusable blocks of code written by the programmer, usually placed after `main()` and called when needed.

```
sections.c

#include <stdio.h>

int square(int x) {           // user-defined
    return x * x;             // function
}

int main() {
    int n = 4;                 // declaration part
    int ans = square(n);       // executable part
    printf("Square = %d", ans);
    return 0;
}
```

Output: Square = 16

Character Set

- **Meaning**

A character denotes any alphabet, digit, or special symbol used to form words and expressions.

- **Letters**

Upper-case A–Z and lower-case a–z.

- **Digits**

All decimal digits 0–9, used in numbers and constants.

- **Special characters**

Symbols such as + - * / = () { } , ; used as operators and punctuation.

- **White space**

Blanks, tabs and newlines that separate tokens but are otherwise ignored.

```
charset.c

#include <stdio.h>

int main() {
    char letter = 'A'; // a letter
    int digit = 7;     // a digit
    char symbol = '+'; // special char
    printf("%c %d %c", letter, digit, symbol);
    return 0;
}
```

Output: A 7 +

Tokens in C

- **Meaning**

The smallest individual units of a C program are called tokens.

- **Six types**

Keywords, Identifiers, Constants, Strings, Operators, Special symbols.

- **Compiler's view**

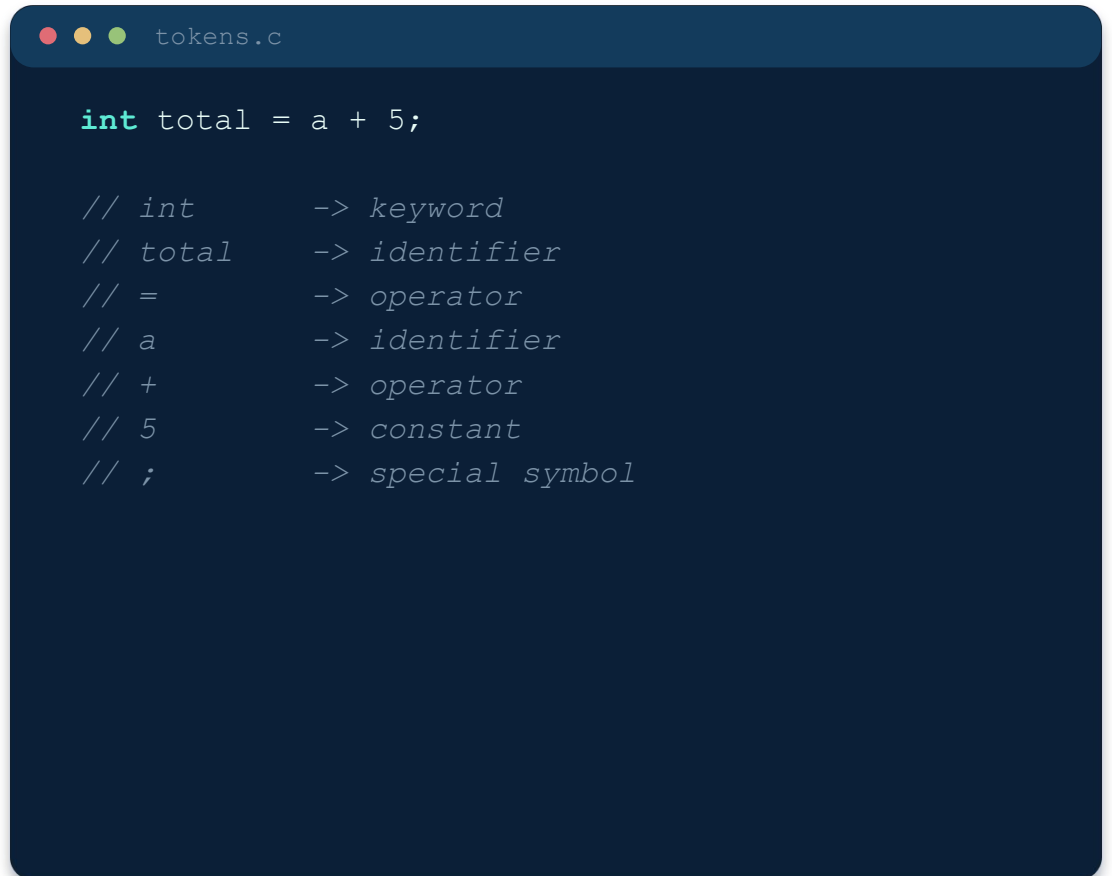
The compiler splits every line into tokens before translating it to machine code.

- **Example**

The statement on the right breaks down into 7 separate tokens.

- **Why it matters**

Recognising tokens helps you read compiler errors and program structure.



```
tokens.c

int total = a + 5;

// int      -> keyword
// total    -> identifier
// =        -> operator
// a        -> identifier
// +        -> operator
// 5        -> constant
// ;        -> special symbol
```

Output: 7 tokens found in this line

Identifiers

- **Meaning**

User-defined names given to variables, functions, and arrays.

- **First character**

Must start with a letter (a-z, A-Z) or an underscore.

- **Rest of the name**

Only letters, digits, and underscores are allowed after that.

- **Case sensitive**

age and Age are two different identifiers in C.

- **Restriction**

A keyword cannot be used as an identifier; only the first 31 characters are checked.

```
identifiers.c

#include <stdio.h>

int main() {
    int age = 19;           // valid
    float _price = 9.5;     // valid
    int Age = 20;           // different from age

    printf("%d %.1f", age, _price);
    return 0;
}
```

Output: 19 9.5

Keywords

- **Meaning**

Reserved words that already have a fixed meaning built into the compiler.

- **Rule**

Always written in lowercase; cannot be redefined or used as identifiers.

- **Count**

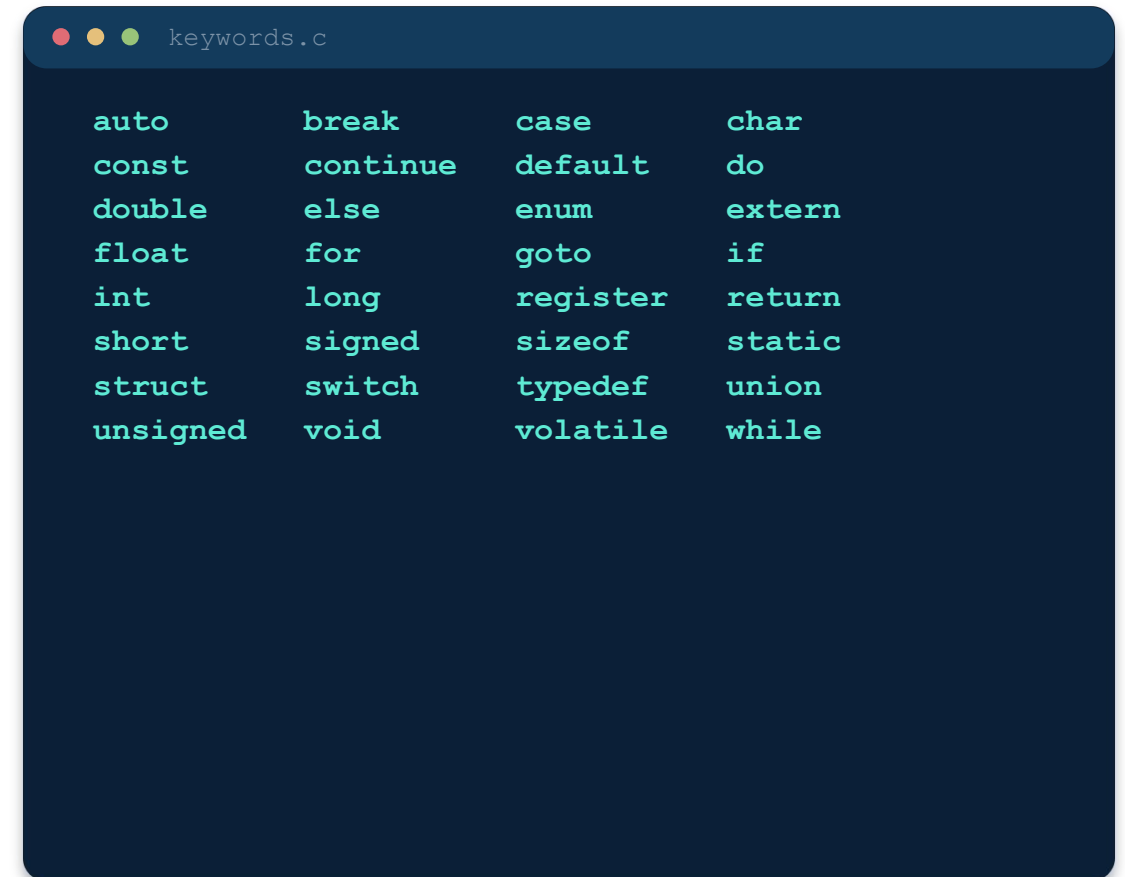
C has exactly 32 keywords — the whole language is built from just these.

- **Building blocks**

They form the basic vocabulary for every statement you write.

- **Examples**

int, if, for, while, return, void, and 26 others.



```
keywords.c

auto      break    case     char
const     continue default  do
double    else      enum      extern
float     for        goto      if
int       long      register  return
short     signed    sizeof    static
struct    switch    typedef   union
unsigned  void      volatile  while
```

Output: 32 reserved words

Constants

- **Meaning**

A constant is a fixed value that does NOT change while the program runs.

- **Integer constant**

Whole numbers, e.g. 10, -5, 2000.

- **Real constant**

Numbers with a decimal point, e.g. 3.14, -0.5.

- **Character constant**

A single character in single quotes, e.g. 'A'.

- **String constant**

Text in double quotes, e.g. "Hello".

Backslash / Escape Constant

Special non-printable characters using backslash

Example : '\n' newline, '\t' tab, '\0' null

```
constants.c

#include <stdio.h>
#define PI 3.14159    // symbolic constant

int main() {
    const int MAX = 100;    // const keyword
    char grade = 'A';
    printf("PI = %f\n", PI);
    printf("MAX = %d\n", MAX);
    printf("Grade = %c", grade);
    return 0;
}
```

Output: PI = 3.14159, MAX = 100, Grade = A

Variables

- **Meaning**

A variable is a named memory location whose value CAN change during execution.

- **Naming rules**

Start with a letter or underscore, only letters/digits/underscore, no spaces, and not a keyword like int or for.

- **Declaration**

You must state the data type before using a variable, e.g. int age;

- **Assignment**

Use = to store a value in a variable, e.g. age = 19;

```
variables.c

#include <stdio.h>

int main() {
    int age = 19;
    float height = 5.8;
    char initial = 'R';

    age = age + 1; // value changes
    printf("Age: %d\n", age);
    printf("Height: %.1f", height);
    return 0;
}
```

Output: Age: 20, Height: 5.8

Expressions

- **Meaning**

A combination of variables, constants, and operators that computes a single value.

- **Arithmetic expression**

$a + b * c \rightarrow$ uses $+$, $-$, $*$, $/$ to calculate a number.

- **Relational expression**

$a > b \rightarrow$ gives a true (1) or false (0) result.

- **Logical expression**

$(a > 0) \&\& (b > 0) \rightarrow$ combines conditions into one true/false result.

- **How it is evaluated**

C follows operator precedence rules, similar to normal maths (BODMAS) — covered in detail later.

```
expression.c

#include <stdio.h>

int main() {
    int a = 10, b = 4, c = 2;
    int result = a + b * c;
    // b*c is done first = 8
    // then a+8 = 18
    printf("Result = %d", result);
    return 0;
}
```

Output: Result = 18

Errors in C

- **Syntax error**

Breaks the grammar rules of C, e.g. a missing semicolon. Caught by the compiler before the program runs.

- **Logical error**

The program runs and compiles, but gives the wrong answer, e.g. using * instead of +.

- **Runtime error**

Happens while the program is running, e.g. dividing a number by zero.

- **Tip**

Always read compiler error messages carefully — they show the exact line number to check.

```
errors.c

#include <stdio.h>
int main() {
    printf("Hi")    // <- Syntax error:
                    // missing semicolon

    int a = 5, b = 2;
    int wrong = a * b; // Logical error
                    // (should be a + b)

    return 0;
}
```

Output: Compiler: error: expected ';'

Operators — Overview

- **Meaning**

Symbols that tell the compiler to perform a specific operation on one or more operands.

- **Coming up next**

The following slides explain each category below in full detail, one at a time.

Category	Example Symbols	Purpose
Arithmetic	+ - * / %	Perform maths calculations
Relational	== != > < >= <=	Compare two values
Logical	&& !	Combine true/false conditions
Assignment	= += -= *= /= %=	Store / update values
Increment & Decrement	++ --	Increase or decrease by 1
Bitwise	& ^ ~ << >>	Operate on individual bits

```
operators.c

#include <stdio.h>
int main() {
    int a = 9, b = 4;
    printf("Sum: %d\n", a + b);
    printf("Equal? %d\n", a == b);
    printf("Both +ve? %d\n", a>0 && b>0);
    a += b;    // assignment operator
    a++;       // increment operator
    printf("a: %d", a);
    return 0;
}
```

Output: Sum: 13, Equal? 0, Both +ve? 1, a: 14

Arithmetic Operators

- **Meaning**

Used to perform basic maths on numeric operands (int, float, double).

Operator	Meaning	Example (a=9, b=4)
+	Addition	$a + b \rightarrow 13$
-	Subtraction	$a - b \rightarrow 5$
*	Multiplication	$a * b \rightarrow 36$
/	Division	$a / b \rightarrow 2$ (integer division)
%	Modulus (remainder)	$a \% b \rightarrow 1$

Note: $9 / 4$ gives 2, not 2.25 — dividing two int values drops the decimal part. Use float to keep it.

```
arithmetic.c

#include <stdio.h>
int main() {
    int a = 9, b = 4;
    printf("Add: %d\n", a + b);
    printf("Sub: %d\n", a - b);
    printf("Mul: %d\n", a * b);
    printf("Div: %d\n", a / b);
    printf("Mod: %d\n", a % b);
    return 0;
}
```

Output: Add:13 Sub:5 Mul:36 Div:2 Mod:1

Relational Operators

- **Meaning**

Compare two values. The result is always 1 (true) or 0 (false).

Operator	Meaning	Example (a=9, b=4)
==	Equal to	a == b → 0
!=	Not equal to	a != b → 1
>	Greater than	a > b → 1
<	Less than	a < b → 0
>=	Greater or equal	a >= 9 → 1
<=	Less or equal	a <= 4 → 0

```
relational.c

#include <stdio.h>
int main() {
    int a = 9, b = 4;
    printf("%d\n", a == b);
    printf("%d\n", a != b);
    printf("%d\n", a > b);

    if (a > b) {
        printf("a is bigger");
    }
    return 0;
}
```

Output: 0, 1, 1, a is bigger

Logical Operators

- **Meaning**

Combine two or more conditions into a single true/false result.

- **Short-circuit rule**

&& stops as soon as one side is false; || stops as soon as one side is true.

Operator	Name	True when...
&&	Logical AND	BOTH sides are true
	Logical OR	AT LEAST ONE side is true
!	Logical NOT	Reverses true to false, and false to true

```
logical.c

#include <stdio.h>
int main() {
    int marks = 78, attendance = 85;

    if (marks >= 40 && attendance >= 75) {
        printf("Eligible for exam");
    } else {
        printf("Not eligible");
    }
    return 0;
}
```

Output: Eligible for exam

Assignment & Increment/Decrement

- **Assignment operators**

Shorthand ways to update a variable using its own current value.

- **++ and --**

Pre (++a) changes the value first, then uses it. Post (a++) uses the current value first, then changes it.

Operator	Same as	Example (a=5)
a += 2	a = a + 2	a becomes 7
a -= 2	a = a - 2	a becomes 3
a *= 2	a = a * 2	a becomes 10
a /= 2	a = a / 2	a becomes 2 (int)

```
increment.c

#include <stdio.h>
int main() {
    int a = 5;
    printf("a++ (post) = %d\n", a++);
    printf("now a = %d\n", a);

    int b = 5;
    printf("++b (pre) = %d\n", ++b);
    printf("now b = %d", b);
    return 0;
}
```

Output: a++ = 5, now a = 6 | ++b = 6, now b = 6

Bitwise Operators

- **Meaning**

Work directly on the individual bits (0s and 1s) of integer values.

Operator	Name	Works by
&	Bitwise AND	1 only if both bits are 1
	Bitwise OR	1 if either bit is 1
^	Bitwise XOR	1 if bits are different
~	Bitwise NOT	Flips every bit
<<	Left shift	Shifts bits left (×2 per shift)
>>	Right shift	Shifts bits right (÷2 per shift)

```
bitwise.c

#include <stdio.h>
int main() {
    int a = 5, b = 3; // 0101, 0011
    printf("AND: %d\n", a & b);
    printf("OR : %d\n", a | b);
    printf("XOR: %d\n", a ^ b);
    printf("LEFT SHIFT: %d", a << 1);
    return 0;
}
```

Output: AND:1 OR:7 XOR:6 LEFT SHIFT:10

Data Types

- **What are data types?**

They tell the compiler what kind of value a variable will store, and how much memory to reserve.

Type	Example	Size*
int	25, -7	4 bytes
float	3.14	4 bytes
char	'A'	1 byte
double	3.14159265	8 bytes
void	(no value)	—

**Sizes are typical on most systems (may vary by compiler).*

```
datatypes.c

#include <stdio.h>
int main() {
    int age = 20;
    float price = 99.5;
    char grade = 'A';
    double pi = 3.14159265;

    printf("%d %.1f %c %lf",
           age, price, grade, pi);
    return 0;
}
```

Output: 20 99.5 A 3.141593

Data Types in C

Category	Type	Size (typical)	Range / Use
Primary	int	2 or 4 bytes	-32,768 to 32,767 (16-bit) or larger
Primary	float	4 bytes	~ ±3.4E-38 to ±3.4E+38, 6 digit precision
Primary	double	8 bytes	Larger range & precision than float
Primary	char	1 byte	-128 to 127 (stores one character)
Primary	void	—	Represents 'no value' (empty return type)
Modifiers	short, long, signed, unsigned	varies	Modify size/range, e.g. unsigned int, long int
Derived	array, pointer, function	varies	Built from primary types
User-defined	struct, union, enum, typedef	varies	Programmer-defined custom types

Declarations

- **Meaning**

A declaration tells the compiler the name and data type of a variable before it is used.

- **Syntax**

data_type variable_name;

- **Multiple variables**

You can declare several variables of the same type in one line, separated by commas.

- **Initialization**

Giving a value at the time of declaration, e.g. `int x = 5;`

```
declare.c

#include <stdio.h>
int main() {
    int x, y, z;           // declaration
    int total = 0;         // declaration +
                           // initialization
    x = 2; y = 3; z = 4;
    total = x + y + z;
    printf("Total = %d", total);
    return 0;
}
```

Output: Total = 9

Input-Output Statements

- **printf()**

Used to display (print) output on the screen.

- **scanf()**

Used to read (accept) input typed by the user.

- **Format specifiers**

%d for int, %f for float, %c for char, %s for string.

- **The & symbol**

scanf needs & before a variable name to store the value at its memory address.

```
io.c

#include <stdio.h>
int main() {
    int age;
    printf("Enter your age: ");
    scanf("%d", &age);    // input
    printf("You are %d years old",
           age);          // output
    return 0;
}
```

Output: Enter your age: 19 -> You are 19 years old

Compound Statements

- **Meaning**

Also called a block. A group of two or more statements enclosed in curly braces {}.

- **Treated as one unit**

The compiler treats the whole block as a single statement, usually after if, else, for, while.

- **Scope**

Variables declared inside a block exist only inside that block.

```
compound.c

#include <stdio.h>
int main() {
    int marks = 75;

    if (marks >= 40) {          // block starts
        printf("Result: Pass\n");
        printf("Marks: %d", marks);
    }                          // block ends
    return 0;
}
```

Output: Result: Pass, Marks: 75

if & if-else Statement

Syntax: `if (condition) { ... } else { ... }`

- **if statement**

Runs its block ONLY when the condition is true. Nothing happens if it is false.

- **if-else statement**

Runs one block if the condition is true, and a different block if it is false.

- **else-if ladder**

Chain several conditions with else if to check them one by one, top to bottom.

```
if_else.c

#include <stdio.h>
int main() {
    int marks = 82;

    if (marks >= 90) {
        printf("Grade: A");
    } else if (marks >= 75) {
        printf("Grade: B");
    } else if (marks >= 40) {
        printf("Grade: C");
    } else {
        printf("Grade: Fail");
    }
    return 0;
}
```

Output: Grade: B

switch Statement

Syntax: `switch (var) { case v1: ...break;
default: ... }`

- **Meaning**

Checks one variable against many possible values (cases), and runs the matching block.

- **break is important**

Without break, execution 'falls through' into the next case, running it too.

- **default**

Runs when no case matches. Similar to the final else in an if-else ladder.

```
switch.c

#include <stdio.h>
int main() {
    int day = 3;
    switch (day) {
        case 1: printf("Monday"); break;
        case 2: printf("Tuesday"); break;
        case 3: printf("Wednesday"); break;
        default:
            printf("Invalid day");
    }
    return 0;
}
```

Output: Wednesday

Conditions

- **Meaning**

An expression that is checked to see if it is true (1) or false (0).

- **Built using**

Relational operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) seen earlier.

- **Combined using**

Logical operators (`&&`, `||`, `!`) to test more than one thing at once.

- **Used in**

if, while, for, and do-while statements to control which code runs.

```
conditions.c

#include <stdio.h>
int main() {
    int age = 20, hasID = 1;

    if (age >= 18 && hasID == 1) {
        printf("Entry allowed");
    } else {
        printf("Entry denied");
    }
    return 0;
}
```

Output: Entry allowed

Operators and Its Precedence

- **What is precedence?**

The order in which operators are evaluated in an expression, just like BODMAS in maths.

- **Same precedence?**

Then associativity decides direction — most operators go left to right.

Order	Operators
1 (highest)	() parentheses
2	* / %
3	+ -
4	< <= > >=
5	== !=
6	&&
7 (lowest)	

```
precedence.c

#include <stdio.h>
int main() {
    int result = 10 + 2 * 5;
    // * runs before + (precedence)
    // 2*5 = 10, then 10+10 = 20

    printf("Result = %d", result);
    return 0;
}
```

Output: Result = 20

Branching: break & continue

- **Meaning**

Both are jump statements used INSIDE loops to change the normal flow.

- **break**

Immediately exits (stops) the nearest loop or switch statement completely.

- **continue**

Skips only the rest of the current cycle, then moves on to the next repetition.

- **Difference**

break ends the loop early; continue just skips one round and keeps looping.

```
break_continue.c

#include <stdio.h>
int main() {
    for (int i = 1; i <= 6; i++) {
        if (i == 5) {
            break;           // stop looping
        }
        if (i % 2 == 0) {
            continue;        // skip even nums
        }
        printf("%d ", i);
    }
    return 0;
}
```

Output: 1 3

Branching: goto Statement

Syntax: `goto label; ... label: statement;`

- **Meaning**

Jumps program control directly to a labeled line, skipping normal top-to-bottom flow.

- **Label**

A name followed by a colon (:) placed before the target statement.

- **Use with caution**

`goto` can make programs hard to read and debug. Loops and functions are usually a better choice.

```
goto.c

#include <stdio.h>
int main() {
    int i = 1;

    start:                // label
        printf("%d ", i);
        i++;
        if (i <= 5) {
            goto start;    // jump back up
        }
    return 0;
}
```

Output: 1 2 3 4 5

Looping: for Statement

Syntax: `for (init; condition; update) { ... }`

- **Meaning**

Repeats a block a fixed, known number of times. Entry-controlled — the condition is checked before every run.

- **Step-by-step flow**

1) init runs once 2) condition is checked 3) if true, body runs 4) update runs 5) repeat from step 2.

- **Best used when**

You already know how many times to repeat, e.g. looping from 1 to 10.

```
for_loop.c

#include <stdio.h>
int main() {
    int sum = 0;
    for (int i = 1; i <= 10; i++) {
        sum = sum + i;
    }
    printf("Sum = %d", sum);
    return 0;
}
```

Output: Sum = 55

Looping: while Statement

Syntax: `while (condition) { ... }`

- **Meaning**

Repeats a block WHILE the condition stays true. Entry-controlled — checked before each run, just like for.

- **Flow**

Check condition → if true, run body → check again → stops the moment it becomes false.

- **Best used when**

You don't know exactly how many repeats are needed, only the condition to stop on.

```
while_loop.c

#include <stdio.h>
int main() {
    int n = 5;
    while (n > 0) {
        printf("%d ", n);
        n--;
    }
    return 0;
}
```

Output: 5 4 3 2 1

Looping: do-while Statement

Syntax: `do { ... } while (condition);`

- **Meaning**

Similar to while, but exit-controlled — the body runs FIRST, and the condition is checked AFTER.

- **Key difference**

A do-while loop always runs at least once, even if the condition is false right from the start.

- **Best used when**

The loop body must execute at least one time, e.g. showing a menu before asking to repeat.

```
do_while.c

#include <stdio.h>
int main() {
    int n = 1;
    do {
        printf("%d ", n);
        n++;
    } while (n <= 3);
    return 0;
}
```

Output: 1 2 3

PRACTICE

Practice Questions

Try writing C programs for each of these on your own.

1

Write a C program to declare two integer variables, take their values from the user, and print their sum using printf() and scanf().

2

Write a C program using an if-else statement to check whether a given number is positive, negative, or zero.

3

Write a C program that uses a for loop to print the multiplication table of a number entered by the user.

4

Write a C program to swap the values of two variables using a third (temporary) variable, and explain each step.

5

Write a C program using a switch statement to print the name of the day of the week for a number entered (1 = Monday, ... 7 = Sunday).